

MCP Server Tool Reference

flowise-questionnaire-system

Generated 2026-08-17

Every example below was tested for real against this server's live production data, using module names the way a real user would type them -- never an internal ID. The one exception is a discrepancy's own ID, used only in the second step of the two routing-diff tools: you never type that from memory, you get it from the first step's own result and pass it straight into the second ask, exactly as shown in those sections. Module names, question names, and counts below reflect this project's real questionnaires at the time this was generated and will drift as the data changes -- if a name below no longer resolves, just ask to list the modules first to see current names.

How this works

This server gives an AI assistant (Claude Desktop, or any other MCP-compatible client) read-only access to your questionnaire data. You don't call tools directly -- you just ask your assistant a plain-language question, the same way you'd ask a colleague. Every tool has a matching prompt shortcut (e.g. typing `"/showModuleSummary"`) that pre-fills a good question for you, but plain conversational asking works just as well.

The one thing worth knowing: names vs. IDs

Every tool that takes a module accepts either its name or its internal ID -- always use the name, exactly as it appears when you list modules. If two modules happen to share an exact name, the assistant will tell you so and show you both IDs to disambiguate, rather than guessing. The one thing you can't substitute a name for is a discrepancy ID (used by the two "detail" tools) -- but you're never expected to know one of those ahead of time; you always get it from the matching report tool's own answer first.

What this server will never do

It's read-only -- it never changes your data, and it never runs the expensive steps (extracting a new upload, building a graph, running a comparison) on its own. If a report says matching hasn't been run yet for a pair of modules, that has to be done first from the Django app itself; the server will tell you plainly rather than guessing at an answer.

Contents

Finding and reading a module

- `list_modules`
- `get_module_summary`
- `list_questions`
- `get_question`

Understanding routing

- `get_routing_edges`
- `trace_variable`
- `get_module_graph`
- `evaluate_edge_condition`

Colectica vs Forsta+ (cross-system check)

- `get_routing_diff_report`
- `get_routing_discrepancy_detail`

Coverage testing

- `get_routing_simulation`

Colectica vs Colectica (wave-to-wave check)

- `get_version_diff_report`
- `get_version_diff_discrepancy_detail`

list_modules

Scope: Everything you have access to

Prompt shortcut: /listModules

Lists every Colectica-format module in the system -- name, status, and how many questions/routing edges it has. Always a safe first move: use it to find the exact name to use in every other tool below.

How to ask

"What questionnaire modules do I have access to?"

What you get back

11 modules, including:
benefits_w18: extracted, 20 questions, 35 routing edges
W18: extracted, 1,803 questions, 4,152 routing edges
W16: extracted, 1,981 questions, 4,465 routing edges
forsta plus: Forsta+ format, extracted, 2,113 questions, 6,561 routing edges
caring_w18: extracted, 9 questions, 16 routing edges

get_module_summary

Scope: One module, in detail

Prompt shortcut: /showModuleSummary

The full status picture for one module: how many questions and edges it has (broken down by conditional/loop/sequential), whether its routing graph has been built, and any extraction error.

How to ask

"Give me the full status summary for benefits_w18."

What you get back

Status: extracted

Questions: 20

Routing edges: 35 total -- 8 conditional, 8 loop, 19 sequential

Graph built: yes

Last updated: 2026-07-02

list_questions

Scope: Every question in a module

Prompt shortcut: /listQuestions

The full question list for one module: name, wording, type, and answer options for each.

How to ask

"List every question in benefits_w18, with type and options."

What you get back

20 questions, including:

INTBENEFIT: single_choice -- intro to the benefits section

BENBASE: multi_choice -- which payments the household currently receives

FICODE: single_choice -- main source of family income

FRVAL: number -- value of that income per week

FRADDL: single_choice -- whether they receive any other payments

get_question

Scope: One question, in full

Prompt shortcut: /showQuestion

Everything about a single named question: wording, type, options, help text, and interviewer instructions.

How to ask

"Show me the full detail for question FICODE in benefits_w18."

What you get back

Type: single_choice

Sequence position: 13th question in the module

Text: asks for the main source of family income

get_routing_edges

Scope: Every routing edge in a module

Prompt shortcut: /listRoutingEdges

The raw routing map for one module -- every source-to-target edge with its type (conditional / loop / sequential) and, for conditional edges, the gating condition.

How to ask

"List every routing edge in benefits_w18."

What you get back

35 edges, including these real conditional ones:

BENBASE -> BENWCA: fires when BenBase = 4

FICODE, FRVAL -> FITAX: fires when FICODE is 2, 3, or 4 AND FRVAL is 0 or more -- a two-variable condition

FICODE -> FRADDL: fires when FICODE is one of a list of income-source codes

trace_variable

Scope: Find every edge that references a variable

Prompt shortcut: /traceVariable

Searches every routing edge (in one module, or across all of them if you don't name one) for a variable, and tells you whether each reference is an "external" variable -- one that isn't actually a question in that module.

How to ask

"Find every routing edge in benefits_w18 that references FICODE."

What you get back

3 matches, all internal (FICODE is a real question in this module):

FICODE, FRVAL -> FITAX: not external

FICODE -> FRADDL: not external

FICODE -> FISEQ: not external

How to ask

"Now search for GRIDVARIABLES.ModeType across the whole W18 module (no specific module given)."

What you get back

122 matches -- this one really is external (GRIDVARIABLES.ModeType isn't a question in W18, it's fed in from outside the module):

GRIDVARIABLES.ModeType -> CHKWEBDOBD: external

GRIDVARIABLES.ModeType -> CHKWEBDOBM: external

This second search omits a module name entirely, so it checks every Colectica module you have access to.

get_module_graph

Scope: The routing graph, visually

Prompt shortcut: /showModuleGraph

The enriched routing graph for a module -- Colectica or Forsta+. Small modules (50 questions or fewer) come back in full; larger ones come back summarized, and you name a specific question to zoom into its neighborhood instead. Ask your assistant to render the diagram, not just describe it.

How to ask

"Show me the full routing graph for benefits_w18, and draw the diagram."

What you get back

20 nodes, 4 real routing paths shown as a diagram, e.g. BENBASE branching to BENCTC or BENWCA depending on the answer, and FICODE branching to FRADDL based on the income-source code.

How to ask

"Now just show me the routing immediately around FICODE in benefits_w18."

What you get back

Neighborhood: NFH, FICODE, FISEQ, FITAX, FRADDL -- FICODE's immediate neighbors, one hop out

Useful once a module is too large to render in full -- zoom into just the question you care about.

evaluate_edge_condition

Scope: "What if" testing for a condition

Prompt shortcut: /evaluateCondition

Evaluates any condition expression against hypothetical answers -- it doesn't need to belong to a real edge, so you can ask exploratory questions about routing logic.

How to ask

"In benefits_w18, if BenBase = 4, does the condition "BenBase = 4" fire?"

What you get back

Result: true -- the condition fires with that answer

get_routing_diff_report

Scope: Colectica vs Forsta+: the full report

Prompt shortcut: /showRoutingDiffReport

Compares a Colectica module's routing against a matched Forsta+ export of the same wave, and lists every place the two systems' routing structurally disagrees. Each row is a lead for a human to check, not a confirmed bug.

How to ask

"Show me the routing-diff report between W18 and forsta plus."

What you get back

Matched: 675 of 1,803 Colectica questions matched to a Forsta+ counterpart (the rest have no confident text/name match -- large modules often have many genuinely unmatched questions, e.g. renamed or restructured between systems)

Discrepancies: 431 total, across 3 types (edges only in Colectica, edges only in Forsta+, and Forsta+ "else" branches Colectica has no equivalent for)

One real example: question PTITLE -> PTITLEX (fires when PTITLE = 6) exists in Colectica but has no matching edge on the Forsta+ side.

This tool never runs the comparison itself -- if it hasn't been run yet for a pair, it says so plainly instead of guessing.

get_routing_discrepancy_detail

Scope: Colectica vs Forsta+: one discrepancy, in full

Prompt shortcut: /showColecticaForstaDiscrepancy

Full detail on one specific discrepancy from the report above -- both systems' routing drawn around the question involved, so you can see exactly what each system does and doesn't do.

How to ask

"From that PTITLE discrepancy in the W18 / forsta plus report, show me the full detail -- both systems' routing around it."

What you get back

Question: PTITLE

Colectica: routes to PTITLEX when PTITLE = 6

Forsta+: has no equivalent edge from PTITLE to PTITLEX

Meaning: either Forsta+ genuinely never routes there (a real gap), or it reaches the same place some other way
this structural check can't see -- worth a human look, not a confirmed bug

The discrepancy ID this needs comes straight from the report tool's own answer -- you ask for the report first, then ask for detail on whichever row stands out; you never need to know an ID ahead of time.

get_routing_simulation

Scope: Does every routing path get tested?

Prompt shortcut: /showRoutingSimulation

Auto-generates a seed answer for every conditional edge in a module and checks whether it actually triggers -- a quick coverage check without having to design test cases by hand.

How to ask

"Run the routing coverage simulation for benefits_w18."

What you get back

Coverage: 8 of 8 auto-generated test cases covered their target edge

Example: seeding BENBASE = 4 successfully triggers the BENBASE -> BENWCA edge, and seeding FRJT = 2 triggers FRJT -> FRJTPN.

get_version_diff_report

Scope: Colectica vs Colectica: comparing two waves

Prompt shortcut: /showVersionDiffReport

Compares two Colectica-format modules against each other -- e.g. two waves of the same questionnaire -- rather than Colectica against Forsta+. Matches questions by name first (since the same variable name usually persists across waves), and additionally flags where a route survives into the newer wave but its gating condition changed.

How to ask

"Compare the routing between W18 (base) and W16 (compare) as two waves of the same questionnaire."

What you get back

Matched: 1,465 of 1,803 W18 questions matched to a W16 counterpart

Discrepancies: 256 total -- edges only in W18, edges only in W16, and edges present in both waves whose firing condition changed

One real example: question NAMEPERM routes to NONAME in both waves, but the condition that triggers it changed between them.

Just like the Colectica/Forsta+ report, this never runs the comparison itself -- it tells you plainly if it hasn't been run yet for this exact pair.

get_version_diff_discrepancy_detail

Scope: Colectica vs Colectica: one discrepancy, in full

Prompt shortcut: /showVersionDiffDiscrepancy

Full detail on one specific version-diff discrepancy -- both waves' routing drawn around the question involved.

How to ask

"From that NAMEPERM discrepancy between W18 and W16, show me the full detail."

What you get back

Question: NAMEPERM

W18 (base): routes to NONAME when NAMEPERM = 1, or when NAME is Don't Know/Refused

W16 (compare): routes to NONAME only when NAMEPERM = 1 -- the NAME condition was dropped

Meaning: both waves agree NONAME is a valid route from here -- what changed is exactly when it fires. This is a textual comparison, not proof the two conditions mean something different in practice -- worth checking whether the change was intentional.

Same workflow as the Colectica/Forsta+ detail tool: ask for the report first, then ask for detail on the specific row you want -- the ID comes from that answer, not from memory.